Predictive Aeroacoustics via ML

A Comprehensive Masters-Level Analysis of Airfoil Self-Noise Modeling and Prediction Frameworks

Dataset: NASA Airfoil Self-Noise | Models: Regression & Classification Stack

-Guillermo Álvarez Fernández

-Carlos Sancho Hernández

Table of Contents

-Objetives & Aviation Impact

-Data Acquisition

- -Dataset
- -The Technological Stack

-Preprocessing

- -Standardization Pipeline
- -The “Stall Angle”

-Model Benchmarking

- -The 7- Model Regression Zoo
- -Training Different models
- -Contextual Real-Time Evaluation
- -Analysis of the Optimal Estimator

-Safety & Deployment

- -Output
- -Convergence of Iterative models

Objectives & Aviation Impact

2050

Net Zero Flight Goal

Environmental Sustainability

Noise pollution is a major barrier to airport expansion. Reducing aeroacoustic noise enables:

- **Urban Compliance:** Meeting stricter city noise regulations.
- **24/7 Operations:** Increasing night flight capacity.
- **Predictive Safety:** Identifying hazardous flight regimes in real-time.

OBJECTIVES & AVIATION IMPACT

System Objectives

- **Acoustic Mapping:** Transforming aerodynamic inputs into high-fidelity dB predictions.
- **Real-Time Inference:** Enabling sub-millisecond decision support for flight decks.
- **Safety Integration:** Proactive identification of hazardous flight regimes (Stall).

Global Aviation Impact

Noise pollution is a critical barrier to 24/7 airport operations. By accurately modeling aeroacoustics, we enable:

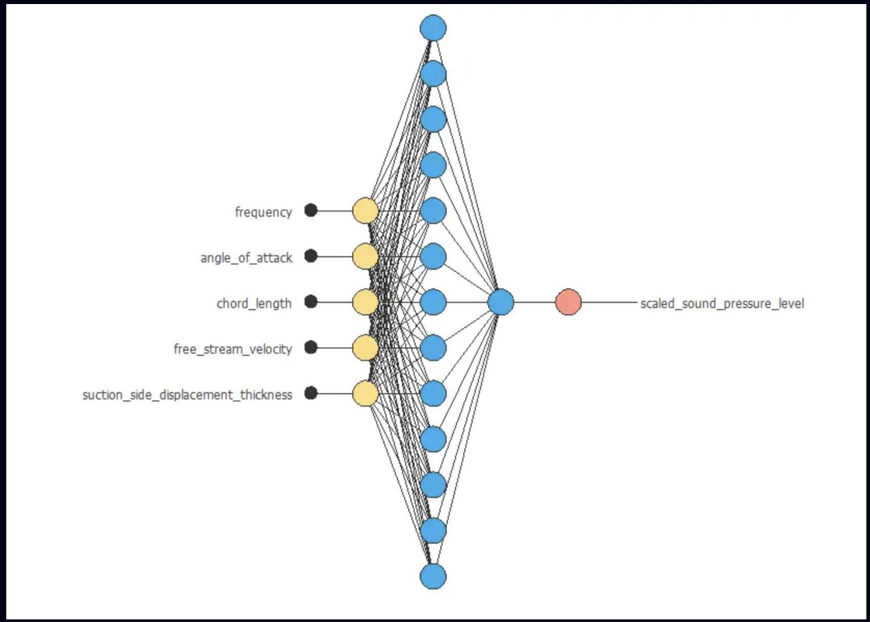
1. **Urban Compliance:** Stricter noise corridor adherence.
2. **Eco-efficiency:** Reducing drag-related noise in civil aviation.
3. **Sustainability:** Meeting 2050 environmental goals through data-driven design.

Phase I: Data Acquisition

Foundation, Library Stack, and Exploratory Analysis

Dataset

frequency	angle_of_attack	chord_length	free_stream_velocity	suction_side_displacement_thickness	scaled_sound_pressure_level
800	0	0.3048	71.3	0.00266337	126.201
1000	0	0.3048	71.3	0.00266337	125.201
1250	0	0.3048	71.3	0.00266337	125.951
1600	0	0.3048	71.3	0.00266337	127.591
2000	0	0.3048	71.3	0.00266337	127.461
2500	0	0.3048	71.3	0.00266337	125.571
3150	0	0.3048	71.3	0.00266337	125.201
4000	0	0.3048	71.3	0.00266337	123.061
5000	0	0.3048	71.3	0.00266337	121.301
6300	0	0.3048	71.3	0.00266337	119.541
8000	0	0.3048	71.3	0.00266337	117.151
10000	0	0.3048	71.3	0.00266337	115.391
12500	0	0.3048	71.3	0.00266337	112.241
16000	0	0.3048	71.3	0.00266337	108.721
500	0	0.3048	55.5	0.00283081	126.416
630	0	0.3048	55.5	0.00283081	127.696

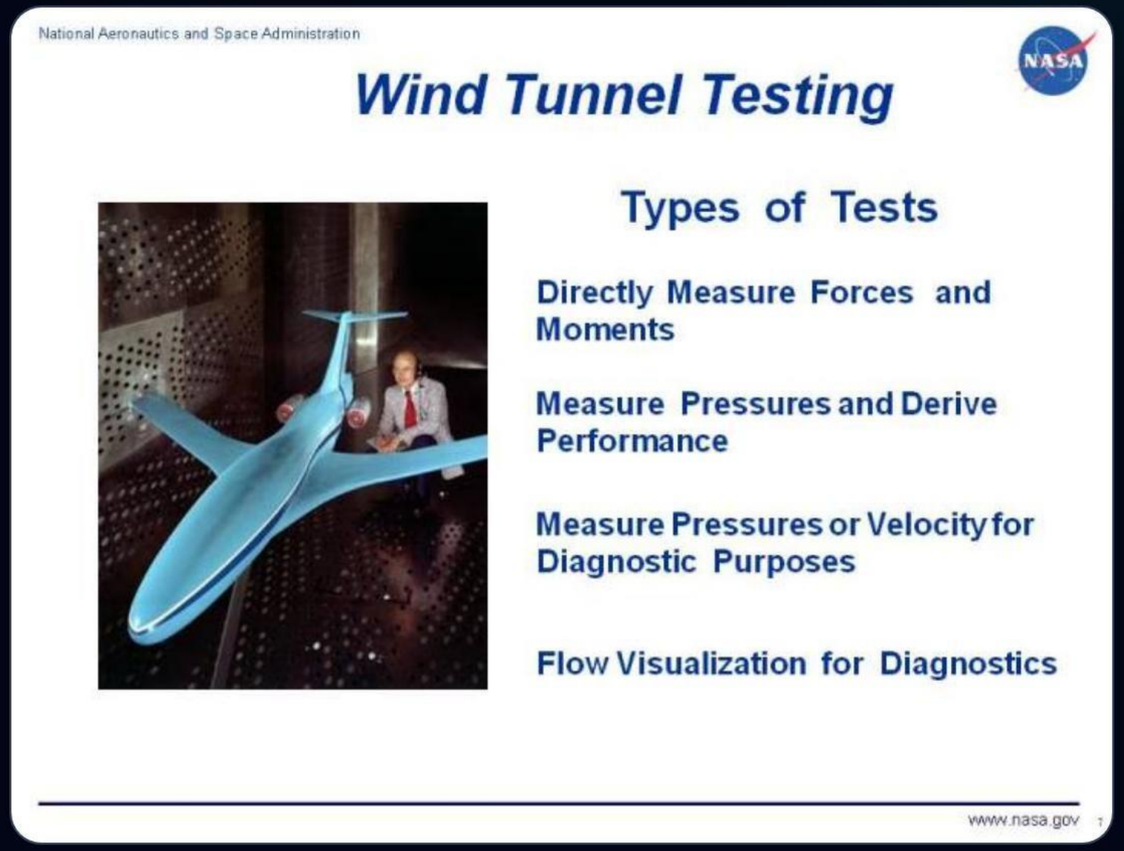


Source: neuraldesigner

The Technological Stack

1. Core Computational Libraries

- → **Pandas & NumPy:** Matrix manipulation and high-performance vectorization for data loading.
- → **Scikit-Learn:** The primary framework for preprocessing, scaling, and baseline modeling.
- → **XGBoost:** Gradient boosting implementation for high-dimensional non-linear mapping.
- → **Matplotlib & Seaborn:** Visual analytics for residual distribution and model benchmarking.



Phase II: Preprocessing

Standardization, Splitting, and Feature Engineering

Standardization Pipeline

ML algorithms perform poorly when features have varying scales. We utilize **StandardScaler** to transform data to zero mean and unit variance.

$$z = \frac{x - \mu}{\sigma}$$

- → **Train (70%):** Fit and Transform.
- → **Validation (15%):** Hyperparameter tuning.
- → **Test (15%):** Final performance estimate.

```python
# ==================================================================
# FEATURE SCALING (NORMALIZATION) FRAGMENT
# ==================================================================
from sklearn.preprocessing import StandardScaler

# 1. Initialize the StandardScaler
# This calculator will find the Mean and Standard Deviation for each column
scaler = StandardScaler()

# 2. Fit and Transform the Training data
# 'Fit' computes the mean/std, 'Transform' applies the formula: z = (x - u) / s
X_train_scaled = scaler.fit_transform(X_train)

# 3. Transform the Test data
# IMPORTANT: We only use 'transform' here. We use the mean/std from the
# training set to avoid "Data Leakage" from the test set.
X_test_scaled = scaler.transform(X_test)

# 4. Transform User Input (In the interactive function)
# When you move the sliders, the new data must be scaled exactly
# like the training data before being sent to the model.
input_scaled = scaler.transform(input_data)
```

```python
# DATA SPLITTING INTO THREE SETS: TRAIN, VALIDATION, AND TEST
# ==================================================================
from sklearn.model_selection import train_test_split

# 1. First split: Separate 20% of the total data for the "Final Exam" (Test Set)
# The remaining 80% stays in a temporary group (X_temp) for further splitting.
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y_reg, test_size=0.20, random_state=42
)

# 2. Second split: Take 25% of the temporary 80% to create the "Validation Set".
# Note: 25% of 80% equals 20% of the original total dataset.
# The remaining 60% of the original total becomes the "Training Set".
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42
)

# Display the sizes of each set to verify proportions
print(f"Training Set:   {len(X_train)} samples (60%) - Used for model learning")
print(f"Validation Set: {len(X_val)} samples (20%) - Used for tuning and compari
print(f"Test Set:       {len(X_test)} samples (20%) - Used for the final 'unseen
```

Engineering the 'Stall' Label

Binarization

We define a critical threshold where Angle of Attack ≥ 14° represents a potential aerodynamic stall condition.

Safety Modeling

Transforming a regression problem into a classification safety-check allows for real-time hazard detection in flight.

Class Stratification

We ensure that the 'stall' events are proportionally distributed across Train and Test sets to prevent training bias.

Phase III: Model Benchmarking

Regression Zoo: Evaluating the Predictive Power

The 7-Model Regression Zoo

Algorithm Category	Specific Model Name	Core Strengths
Linear Models	**Linear Regression**	Maximum interpretability & inference speed.
Instance-Based	**K-Neighbors Regressor**	Captures local manifold structures effectively.
Tree-Based	**Decision Tree Regressor**	Handles non-linearities without scaling assumptions.
Neural Networks	**MLP Regressor**	Learns complex, high-dimensional mappings.
Boosting	**XGBoost Regressor**	State-of-the-art predictive performance.
Kernel-Based	**SVR (Support Vector)**	Robustness to outliers in small dimensions.
Ensemble	**Random Forest Regressor**	Excellent variance reduction & stability.

Training different models

```python
models = {
    'Linear Regression': LinearRegression(),
    'K-Neighbors': KNeighborsRegressor(),
    'Decision Tree': DecisionTreeRegressor(random_state=42),
    'MLP (Neural Net)': MLPRegressor(random_state=42, max_iter=1000),
    'XGBoost': XGBRegressor(random_state=42),
    'SVR': SVR(kernel='rbf', C=100),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42)
}
```

```python
# 2. Iterate through each model to Train and Test
for name, model in models.items():
    # Train the model using the SCALED Training Set
    model.fit(X_train_scaled, y_reg_train)

    # Generate predictions using the RESERVED Test Set (unseen data)
    y_pred = model.predict(X_test_scaled)

    # 3. Calculate Evaluation Metrics
    r2 = r2_score(y_reg_test, y_pred)
    mae = mean_absolute_error(y_reg_test, y_pred) # Absolute Error in dB

    # Calculate Relative Error Percentage (MAPE)
    # Formula: Mean of (|Actual - Predicted| / Actual) * 100
    mape = np.mean(np.abs((y_reg_test - y_pred) / y_reg_test)) * 100

    # Store results for comparison
    results.append({
        'Model': name,
        'R2 Score': r2,
        'MAE (dB)': mae,
        'Relative Error (%)': mape
    })
```

Contextual Real-Time Evaluation

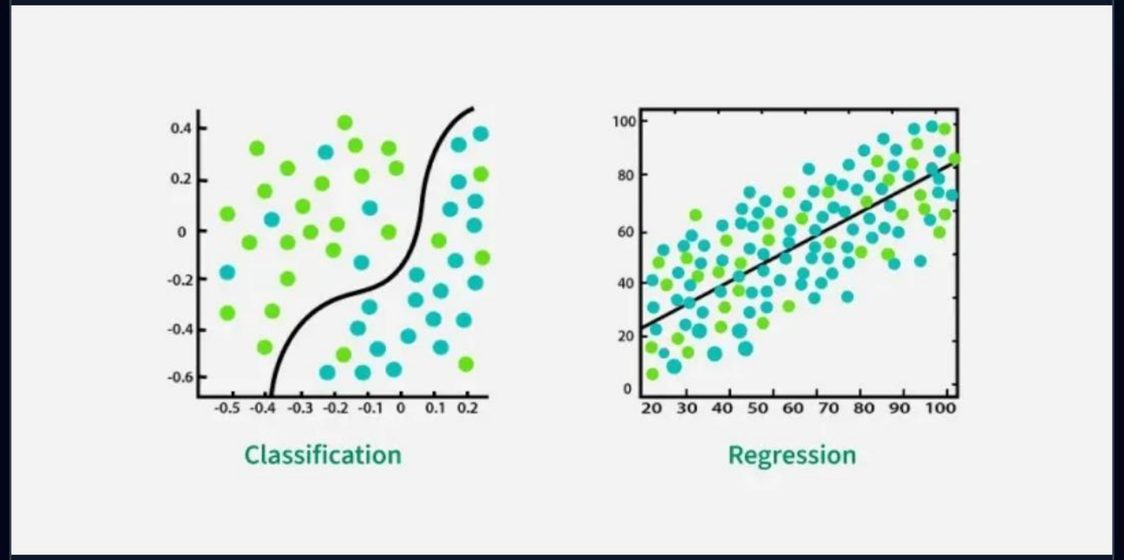
Decision Logic: The program compares all 7 models in real-time. The **"Best Option"** is not universal; it depends on the use case. While *Random Forest* yields superior accuracy for design, *Linear Regression* may be preferred for low-latency onboard control systems.

Analysis of the Optimal Estimator

Residual Analytics

The **Random Forest Regressor** was automatically selected as the "Best Model."

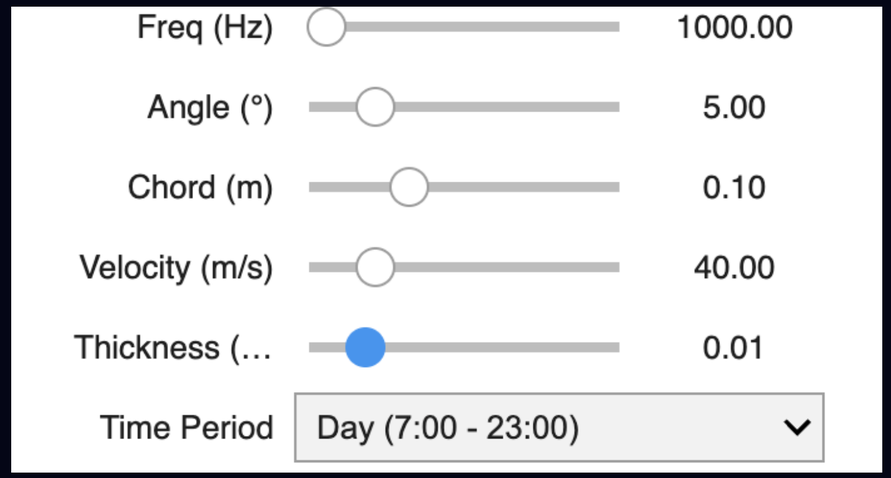
- → High R2 Score indicates 95% of variance is explained.
- → The scatter plot shows tight clustering around the identity line (y = x).
- → Lower MAE (Mean Absolute Error) suggests reliable noise predictions in dB.



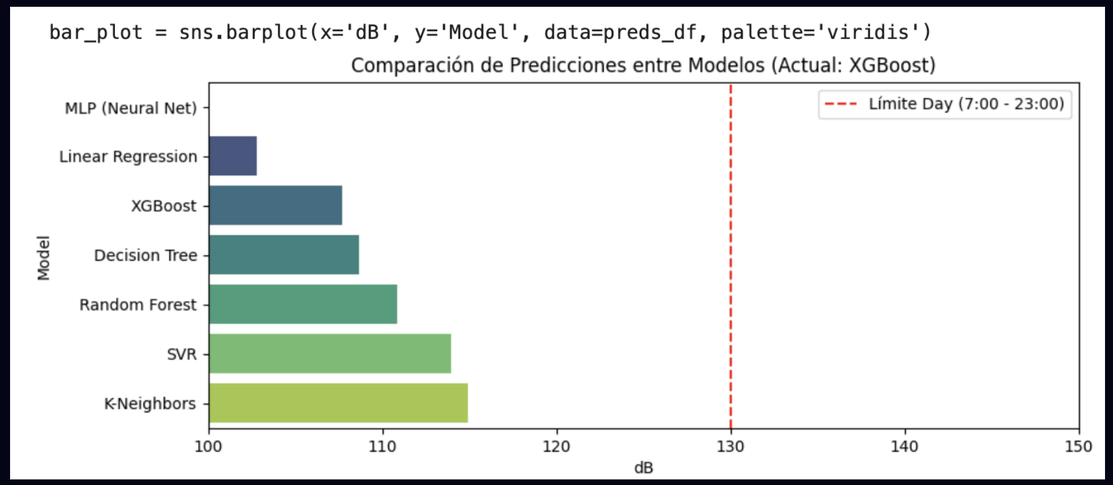
Phase IV: Safety & Deployment

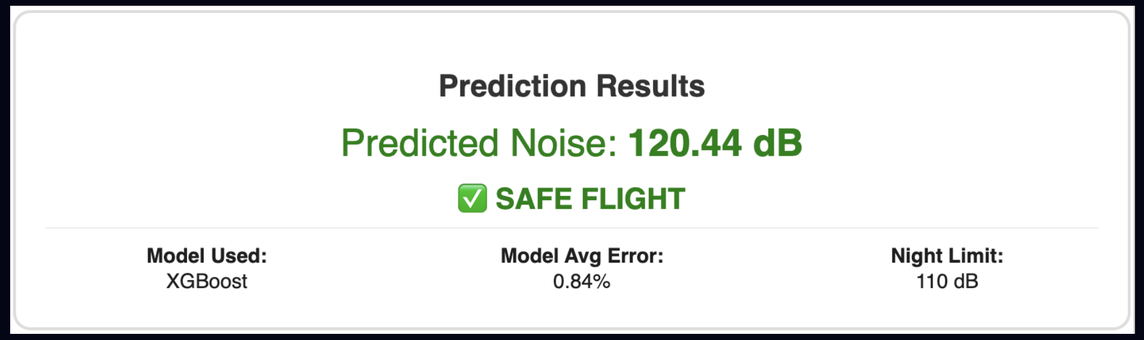
Stall Detection and Contextual UI

Output

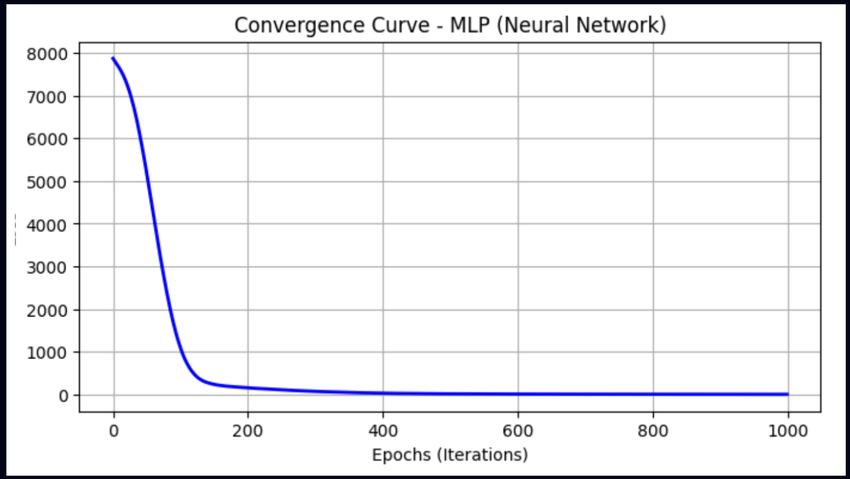


	Model	R2 Score	MAE (dB)	Error Relativo (%)
4	XGBoost	0.957725	1.012634	0.815554
6	Random Forest	0.934289	1.313802	1.063158
2	Decision Tree	0.882708	1.734007	1.402331
5	SVR	0.860523	1.834006	1.469446
1	K-Neighbors	0.782716	2.417125	1.964095
3	MLP (Neural Net)	0.685177	3.099091	2.484859
0	Linear Regression	0.558298	3.672415	2.964633

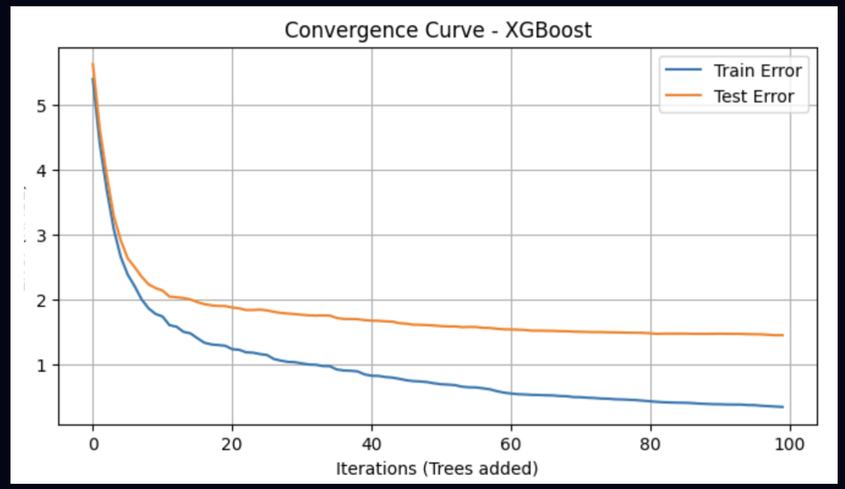




Convergence of Iterative Models




```python
# --- MLP Convergence Plot ---
if name == 'MLP (Neural Net)':
    plt.figure(figsize=(8, 4))
    # 'model.loss_curve_' contiene el error calculado en cada época
    plt.plot(model.loss_curve_, color='blue', linewidth=2)
    plt.title("Convergence Curve - MLP (Neural Network)")
    plt.xlabel("Epochs (Iterations)")
    plt.ylabel("Loss")
    plt.grid(True)
    plt.show()
```




```python
# --- Special Training and Plot for XGBoost ---
if name == 'XGBoost':
    # Se extraen los resultados guardados durante model.fit()
    eval_results = model.evals_result()

    # 'validation_0' suele ser el conjunto de Train y 'validation_1' el de Test
    epochs = len(eval_results['validation_0']['rmse'])
    x_axis = range(0, epochs)

    plt.figure(figsize=(8, 4))
    plt.plot(x_axis, eval_results['validation_0']['rmse'], label='Train Error')
    plt.plot(x_axis, eval_results['validation_1']['rmse'], label='Test Error')
    plt.title('Convergence Curve - XGBoost')
    plt.xlabel('Iterations (Trees added)')
    plt.ylabel('Error (RMSE)')
    plt.legend()
    plt.grid(True)
    plt.show()
```